Atividades – OpenMP Sections e Tasks

Pedro de Souza Zequi – 10419805
Repositório: github.com/PedroSZequi/CompParalelaOpenMPTasksSections

1. Introdução

Este relatório apresenta as três atividades sobre as diretivas sections e task do OpenMP. Em cada uma, o código inicial foi modificado conforme o desafio, e as duas versões foram comparadas em clareza e desempenho.

2. Ambiente e metodologia

- Máquina: GitHub Codespaces, AMD EPYC 7763, 2 núcleos físicos (4 threads de hardware).
- Compilação: gcc -O3 -Wall -fopenmp, feita pelo script compilar.sh.
- Medição: o script medir.sh roda cada programa com 1, 2, 3 e 4 threads (OMP_NUM_THREADS), 5 vezes cada, e calcula a média do tempo medido com omp_get_wtime(). Antes das 5 execuções, faz 1 execução de aquecimento, que é descartada.

```
@PedroSZequi ➜ /workspaces/CompParalelaOpenMPTasksSections (main) $ nproc
4
@PedroSZequi ➜ /workspaces/CompParalelaOpenMPTasksSections (main) $ lscpu | grep -E "Model name|Socket|Core|Thread"
Model name:                           AMD EPYC 7763 64-Core Processor
Thread(s) per core:                   2
Core(s) per socket:                   2
Socket(s):                            1
@PedroSZequi ➜ /workspaces/CompParalelaOpenMPTasksSections (main) $
```

Figura 1 – Ambiente de execução

```
@PedroSZequi ➜ /workspaces/CompParalelaOpenMPTasksSections (main) $ cat compilar.sh
#!/bin/bash
# Compila todas as versoes das tres atividades
# Uso: ./compilar.sh
set -e
CFLAGS="-O3 -Wall -fopenmp"

gcc $CFLAGS atividade1/analise_inicial.c      -o atividade1/analise_inicial      -lm
gcc $CFLAGS atividade1/analise_sections.c     -o atividade1/analise_sections     -lm
gcc $CFLAGS atividade2/simulacao_inicial.c    -o atividade2/simulacao_inicial
gcc $CFLAGS atividade2/simulacao_sections.c   -o atividade2/simulacao_sections
gcc $CFLAGS atividade3/contalinhas_inicial.c  -o atividade3/contalinhas_inicial
gcc $CFLAGS atividade3/contalinhas_tasks.c    -o atividade3/contalinhas_tasks
gcc $CFLAGS atividade3/contalinhas_tasks_v2.c -o atividade3/contalinhas_tasks_v2

echo "Compilacao concluida:"
ls -1 atividade1/analise_inicial atividade1/analise_sections \
      atividade2/simulacao_inicial atividade2/simulacao_sections \
      atividade3/contalinhas_inicial atividade3/contalinhas_tasks \
      atividade3/contalinhas_tasks_v2
@PedroSZequi ➜ /workspaces/CompParalelaOpenMPTasksSections (main) $ ./compilar.sh
Compilacao concluida:
atividade1/analise_inicial
atividade1/analise_sections
atividade2/simulacao_inicial
atividade2/simulacao_sections
atividade3/contalinhas_inicial
atividade3/contalinhas_tasks
atividade3/contalinhas_tasks_v2
@PedroSZequi ➜ /workspaces/CompParalelaOpenMPTasksSections (main) $
```

Figura 2 – Compilação de todas as versões

3. Atividade 1 – Análise estatística (sections)

Alterações

O código inicial calcula o mínimo, o máximo, a soma e a soma dos quadrados em três laços parallel for. Na nova versão, os três cálculos ficaram em uma única região parallel sections, com uma section para cada um:

```
#pragma omp parallel sections private(i) \
      reduction(min:min_val) reduction(max:max_val) \
      reduction(+:sum) reduction(+:sum_sq)
{
   #pragma omp section   // minimo
   #pragma omp section   // maximo
   #pragma omp section   // soma e soma dos quadrados
}
```

As reduções combinam os resultados de cada section no final. O private(i) é necessário porque a variável i foi declarada fora da região paralela. Sem ele, as sections compartilhariam o mesmo contador.

Execução e medição

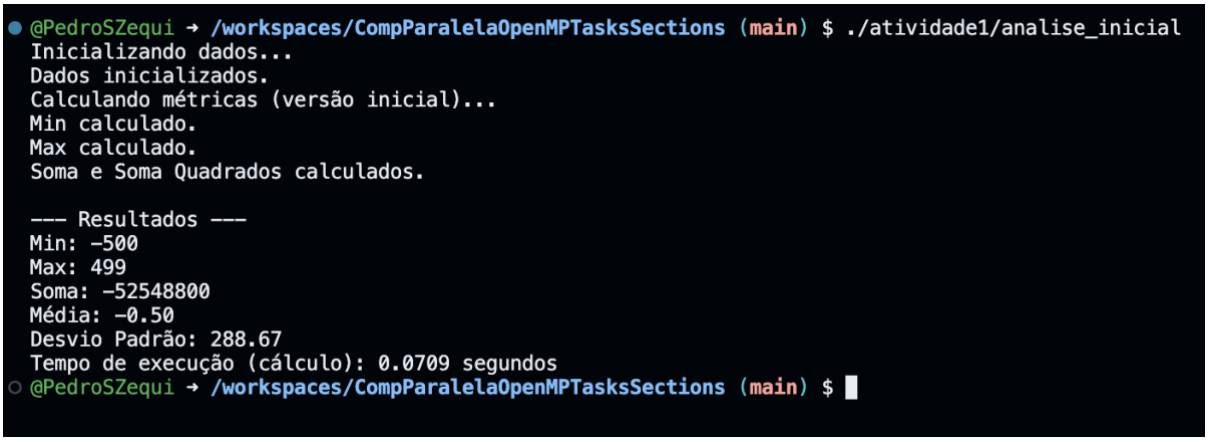


Figura 3 – Versão inicial

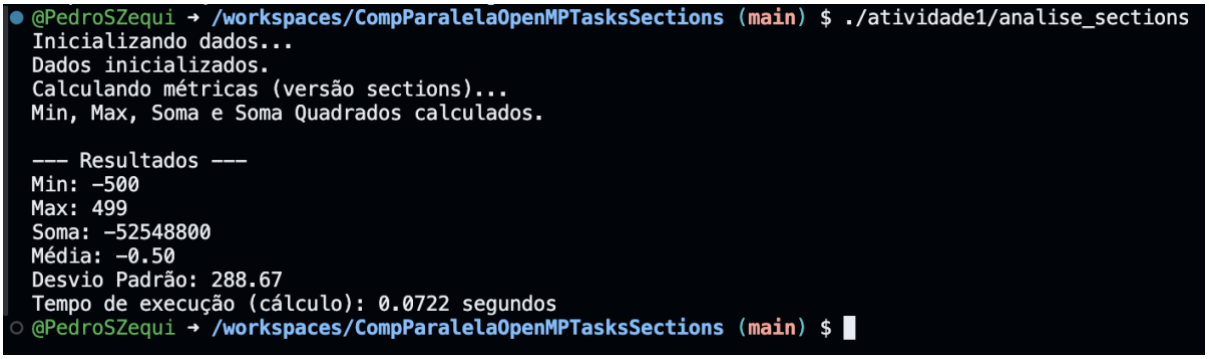


Figura 4 – Versão com sections (mesmos resultados)

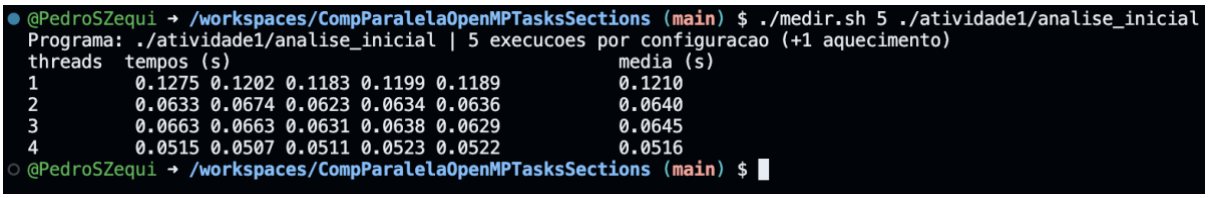


Figura 5 – Medição da versão inicial

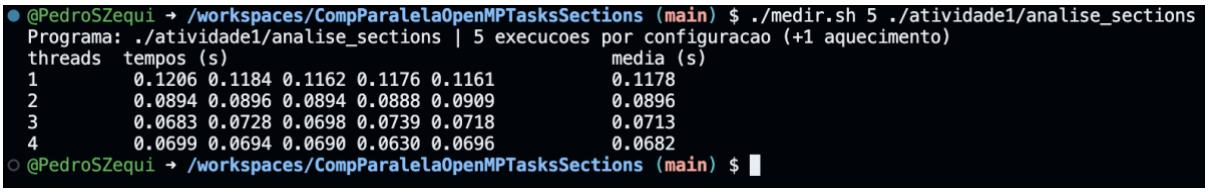


Figura 6 – Medição da versão com sections

Tabela 1 – Atividade 1 – tempo médio (s)

Threads	Inicial	Sections
1	0,1210	0,1178
2	0,0640	0,0896
3	0,0645	0,0713
4	0,0516	0,0682

Análise

Com 1 thread, as duas versões empatam. Com mais threads, a versão inicial é mais rápida. No sections, cada cálculo é feito inteiro por uma única thread, então no máximo 3 threads trabalham ao mesmo tempo, e cada uma percorre o vetor todo. Já o parallel for divide cada laço entre todas as threads.

Quanto à clareza, o sections deixa explícitos os três cálculos independentes, mas o código fica mais longo. Para este problema, o parallel for é mais simples e mais rápido.

4. Atividade 2 – Simulação de partículas (sections)

Alterações

O código inicial atualiza a posição e depois a velocidade das partículas, em dois laços parallel for. Na nova versão, cada laço virou uma section, e as duas rodam ao mesmo tempo:

```
#pragma omp parallel sections private(i)
{
   #pragma omp section   // posicao: y += vy * DT  (le vy)
   #pragma omp section   // velocidade: vy += g * DT (escreve vy)
}
```

Como as duas sections usam vy ao mesmo tempo (uma lê e a outra escreve), existe uma condição de corrida. Para verificar isso, o programa guarda os valores antigos de y e vy e, depois do passo, conta quantas partículas tiveram a posição calculada com a velocidade nova.

Execução e medição

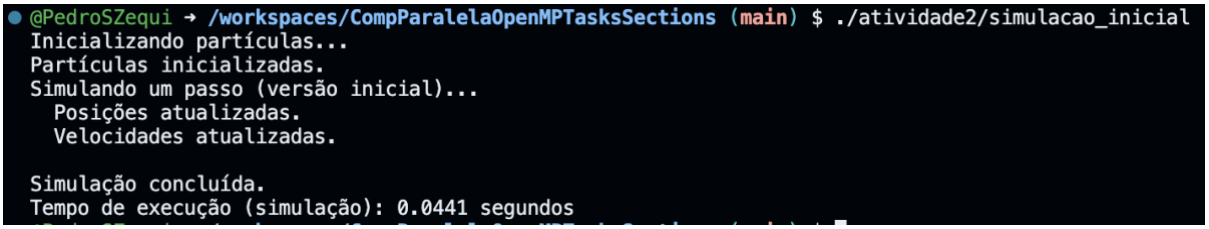


Figura 7 – Versão inicial

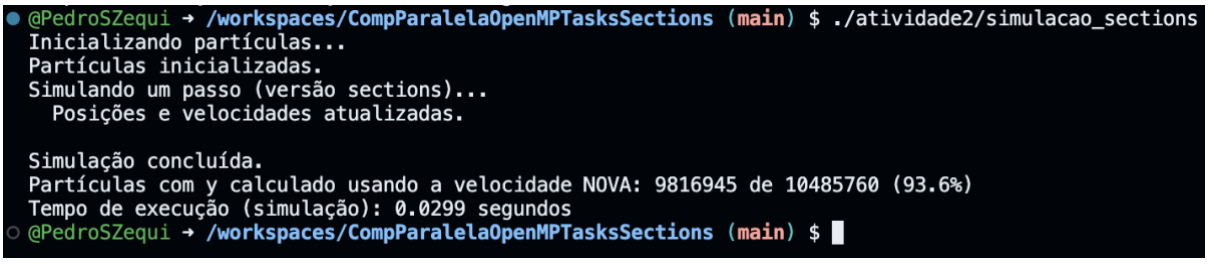


Figura 8 – Versão com sections: 93,6% das partículas usaram a velocidade nova

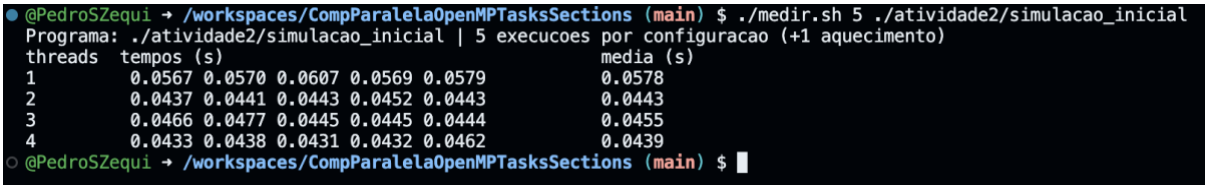


Figura 9 – Medição da versão inicial

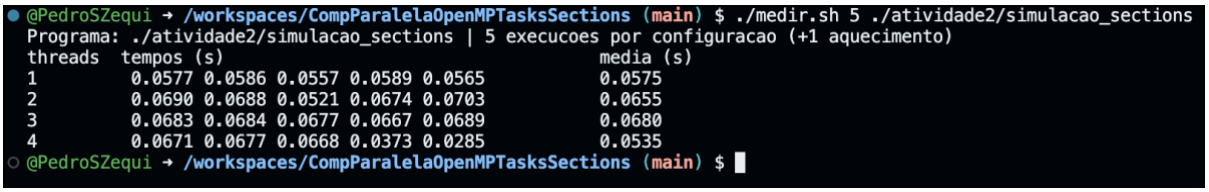


Figura 10 – Medição da versão com sections

Tabela 2 – Atividade 2 – tempo médio (s)

Threads	Inicial	Sections
1	0,0578	0,0575
2	0,0443	0,0655
3	0,0455	0,0680
4	0,0439	0,0535

Análise

A versão com sections deu resultado diferente da original: 93,6% das partículas usaram a velocidade já atualizada. Isso acontece porque a section da velocidade é mais leve e passa à frente da outra. Ou seja, as duas tarefas não são realmente independentes.

Em desempenho, a versão inicial melhora pouco com mais threads, porque o limite é a velocidade da memória. A versão com sections ficou, na maioria das vezes, mais lenta e com tempos instáveis. Uma explicação provável é que y e vy de cada partícula ficam na mesma linha de cache, e as duas threads escrevem nela ao mesmo tempo.

A melhor solução seria um único parallel for que, para cada partícula, atualiza primeiro a posição e depois a velocidade.

5. Atividade 3 – Contagem de linhas (tasks)

Alterações

O código inicial percorre os diretórios de forma recursiva e sequencial. Na nova versão, cada subdiretório e cada arquivo .c/.h encontrado vira uma tarefa:

```
if (diretorio) {
   #pragma omp task firstprivate(fullpath)
   { traverse_directory(fullpath); free(fullpath); }
} else if (arquivo .c ou .h) {
   #pragma omp task firstprivate(fullpath)
   { long long linhas = count_lines_in_file(fullpath);
     #pragma omp atomic
     total_lines_global += linhas;
     free(fullpath); }
}

#pragma omp parallel   // no main
#pragma omp single
traverse_directory(start_dir);
```

- O total de linhas é somado em um contador global com atomic, a forma mais simples de juntar os resultados das tarefas.
- Cada tarefa recebe a própria cópia do caminho (firstprivate) e libera essa memória ao terminar.
- Não é preciso taskwait: a barreira no fim da região parallel garante que todas as tarefas terminaram.
- No código inicial, a primeira linha foi trocada para #define _GNU_SOURCE, para eliminar um aviso de compilação do asprintf.

Execução e medição

O diretório usado nos testes foi /usr/include.

```
@PedroSZequi ➜ /workspaces/CompParalelaOpenMPTasksSections (main) $ ./atividade3/contalinhas_inicial /usr/include
Contando linhas em .c/.h a partir de: /usr/include (versão inicial sequencial)

--- Resultado ---
Total de linhas encontradas: 2405445
Tempo de execução: 0.5335 segundos
@PedroSZequi ➜ /workspaces/CompParalelaOpenMPTasksSections (main) $
```

Figura 11 – Versão inicial

```
@PedroSZequi ➜ /workspaces/CompParalelaOpenMPTasksSections (main) $ ./atividade3/contalinhas_tasks /usr/include
Contando linhas em .c/.h a partir de: /usr/include (versão com tasks, 4 threads)

--- Resultado ---
Total de linhas encontradas: 2405445
Tempo de execução: 0.3018 segundos
@PedroSZequi ➜ /workspaces/CompParalelaOpenMPTasksSections (main) $
```

Figura 12 – Versão com tasks (mesmo total)

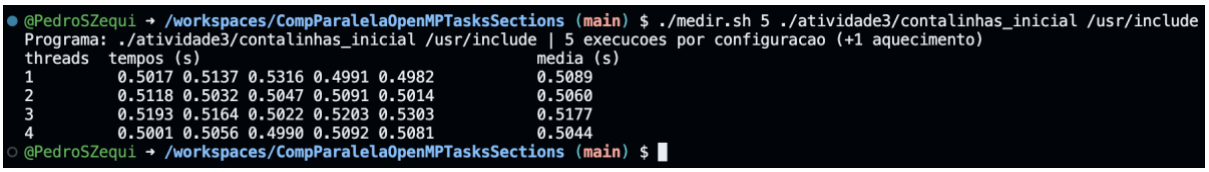


Figura 13 – Medição da versão inicial

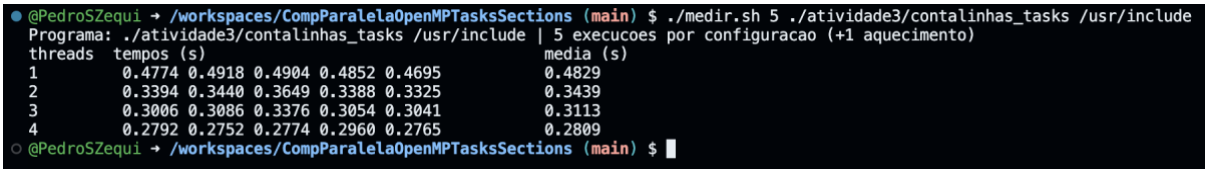


Figura 14 – Medição da versão com tasks

Versão v2 (getc_unlocked)

A versão com tasks ganhou pouco com mais threads. O motivo é que, com várias threads, a função fgetc trava e destrava o arquivo a cada caractere lido. Na versão v2, fgetc foi trocado por getc_unlocked, que lê sem travar. A troca é segura porque cada arquivo é lido por uma única tarefa.



Figura 15 – Diferença entre as versões e execução da v2

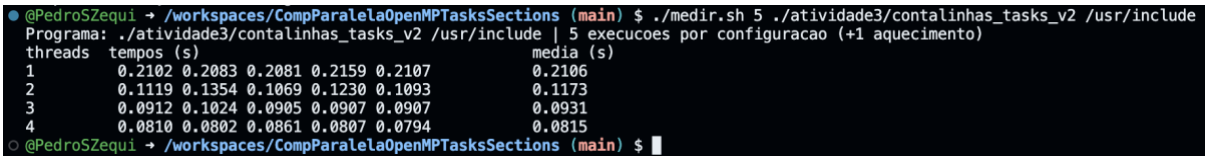


Figura 16 – Medição da versão v2

Tabela 3 – Atividade 3 – tempo médio (s)

Threads	Inicial	Tasks	Tasks v2
1	0,509	0,483	0,211
2	0,506	0,344	0,117
3	0,518	0,311	0,093
4	0,504	0,281	0,082

Análise

A versão inicial é sequencial e fica em cerca de 0,51 s com qualquer número de threads. A versão com tasks chega a 0,28 s com 4 threads, e a v2 a 0,082 s, cerca de 6 vezes mais rápida que a inicial.

Quanto à clareza, a versão com tasks mantém a estrutura recursiva do código original, com poucas mudanças. As tasks combinam bem com este problema, porque não se sabe de antemão quantos arquivos e pastas existem.

6. Conclusão

O sections funciona melhor com poucas tarefas, fixas e realmente independentes. Nas Atividades 1 e 2, ele foi pior que o parallel for e, na Atividade 2, ainda gerou resultados errados. O task foi a melhor escolha para a Atividade 3, que é recursiva e irregular, principalmente depois de remover o custo das travas do fgetc. Em resumo, a melhor diretiva depende da estrutura do problema, e é importante medir os resultados em vez de supor.